SpaceB&B

Relazione del progetto

Ingegneria del Software
Università degli Studi di Ferrara

Tommaso Pizzolato 196187
Matteo Tosi 195958

Progetto congiunto con Basi di Dati e Ingegneria dei Sistemi Web

Oggetto del progetto

Piattaforma web per affitti brevi tra privati, sul modello di Airbnb. Chi si registra può essere host (pubblica) e guest (cerca, prenota, recensisce).

12	29	24
conversazioni funzionali	requisiti funzionali	regole di business

Oltre agli alloggi: noleggio auto, esperienze, spese di viaggio, guida alle città, proposte di vacanza e pannello admin.

Stack: Spring Boot (Java 17), JPA, JWT, MySQL 8.0, HTML/CSS/JavaScript.

Distribuzione del team (ruoli)

T Tommaso Pizzolato

Matricola 196187

41 SP

Alloggi: ricerca, preferiti, spese
Conv. 2, 3, 6 · 17 SP

Ricerca e noleggio auto
Conv. 7 · 9 SP

Scopri la città e amministrazione
Conv. 11, 12 · 15 SP

Anche: Backend, frontend e database in generale

M Matteo Tosi

Matricola 195958

39 SP

Account e recensioni
Conv. 1, 5 · 11 SP

Area host degli alloggi
Conv. 4 · 11 SP

Veicoli ed esperienze (gestione e prenotazione)
Conv. 8, 9, 10 · 17 SP

Anche: Design system, login, homepage e pagamento

Come abbiamo diviso il lavoro

Ognuno copre 3 macro-funzioni complete (frontend, backend, database). Assegnazioni e story points (SP) sono quelli di Trello.

Evidenze delle cerimonie

Gli incontri del team durante il progetto.

1

Sprint planning

Scelta delle conversazioni da realizzare e stima in story points.

2

Daily

Breve aggiornamento su cosa è fatto, cosa si fa, cosa blocca.

3

Sprint review

Verifica di quanto realizzato rispetto ai task su Trello.

4

Retrospettiva

Cosa ha funzionato, cosa migliorare per lo sprint dopo.

Estrazione del disegno funzionale (da Miro)



La board Miro

- User journey di guest e host
- Flusso di ricerca e prenotazione
- Architettura a 3 livelli
- Mappa delle 12 conversazioni, scomposte in frontend, backend e database

Da ogni attività della board sono nati i task su Trello.

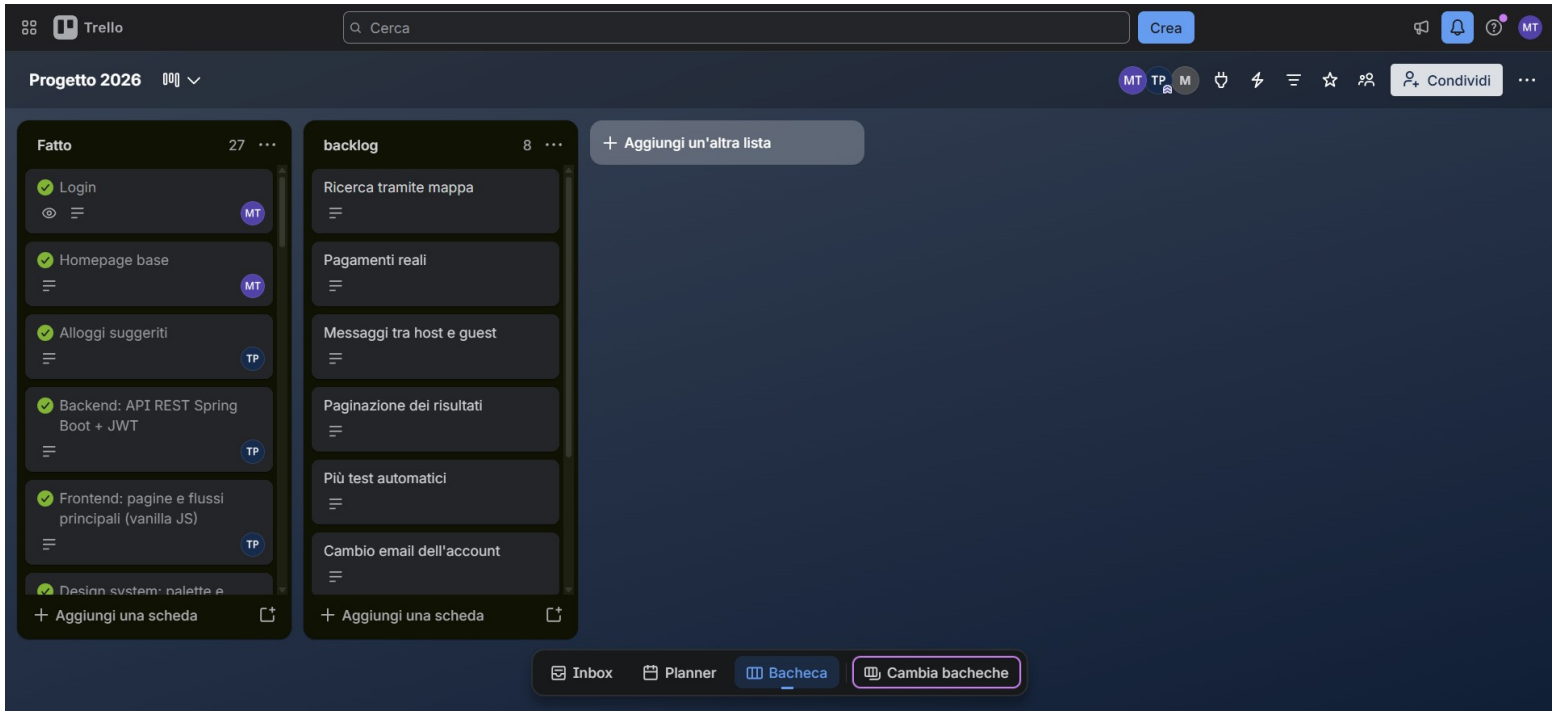
Link Miro in LINKS.txt

Screenshot di Trello

27
card in Fatto

8
idee in backlog

80
story points totali



Come l'abbiamo usato

- Ogni conversazione scomposta in task di frontend, backend e database
- Stima in story points: un CRUD semplice vale 2 SP (il pivot)
- Carico diviso: 41 SP a Tommaso, 39 a Matteo
- Le idee future sono nel backlog

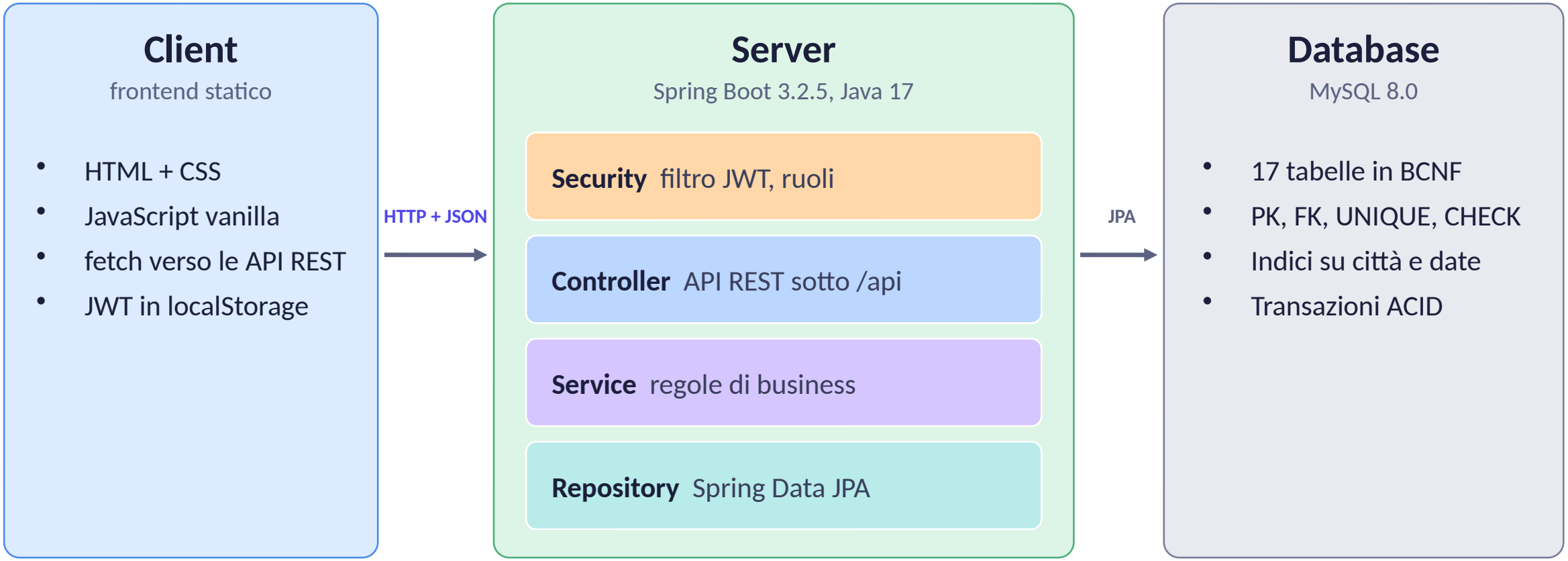
Test cases

ID	Scenario	Input	Esito atteso	Regola
TC-01	Check-in dopo il check-out	check-in 10/11, check-out 08/11	400, date non valide	VR-01
TC-02	Date già occupate	12-14/11 con prenotazione attiva sul 10-15/11	400, date occupate	VR-02
TC-03	Prenotare un proprio alloggio	guest uguale all'host	400	VR-03
TC-04	Ospiti oltre la capienza	numOspiti > ospiti massimi	400	VR-04
TC-05	Prenotazione valida	3 notti a 80 € a notte	201, IN_ATTESA, totale 240 €	VR-05
TC-06	Annullare una prenotazione completata	stato COMPLETATA	400, stato non valido	VR-14

Test implementati

6 casi implementati in PrenotazioneServiceTest (JUnit 5 + Mockito, senza database).

Indicazione del disegno architetturale scelto



REST

API stateless sotto /api, accesso con JWT

MVC

Model: entità JPA · Controller: REST · View: frontend

Frontend e backend separati

comunicano solo via HTTP

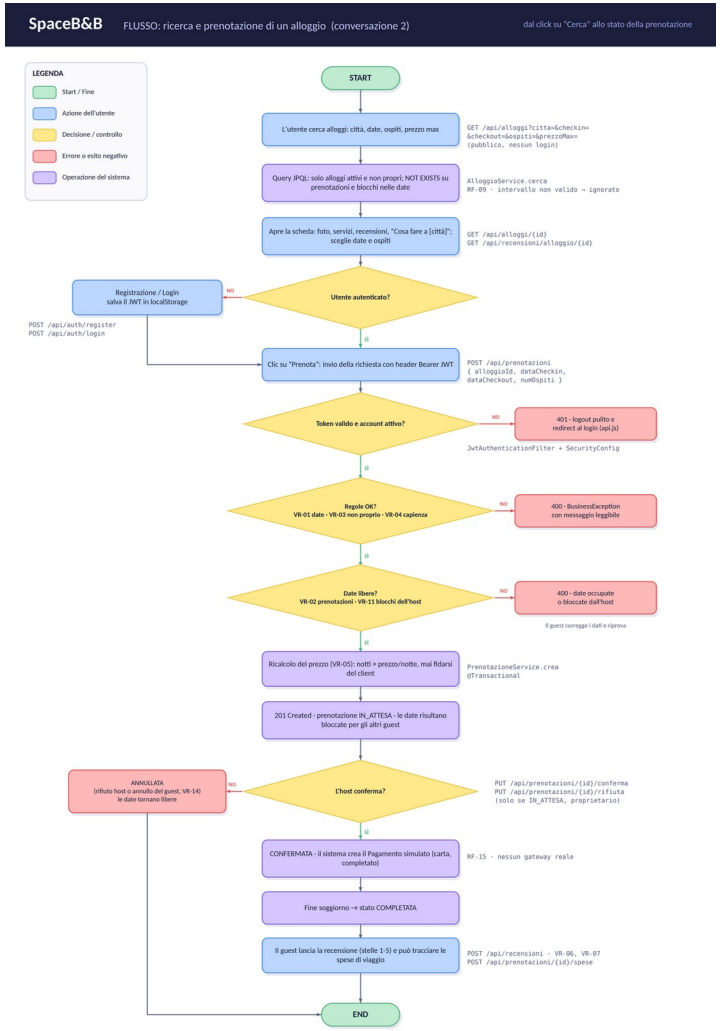
Funzionalità implementate (elenco)

1 Registrazione, accesso e profilo RF-01, 02, 03, 17 — Media	**2** Ricerca e prenotazione alloggio RF-09 – 12 — Alta	**3** Preferiti RF-14 — Bassa	**4** Gestione annuncio (host) RF-04 – 08, 15 — Alta
5 Annullamento e recensione RF-13, 16 — Media	**6** Registro spese di viaggio RF-26 — Bassa	**7** Ricerca e noleggio veicolo RF-23 — Alta	**8** Gestione veicolo (host) RF-22, 23 — Media
9 Prenotazione esperienza RF-25 — Alta	**10** Gestione esperienza RF-24 — Media	**11** Scopri la città e Suggeriti RF-27, 29 — Alta	**12** Pannello di amministrazione RF-18 – 21, 28 — Alta

Usabilità: layout responsive, caricamento a scheletro, messaggi di errore chiari, stati vuoti, pagine riservate protette.

12 conversazioni: 6 di complessità alta, 4 media, 2 bassa. RF = requisito funzionale.

Dettaglio di una funzionalità



Ricerca e prenotazione di un alloggio

Cerchi, scegli le date e prenoti. Il server controlla le regole e calcola il prezzo; l'host conferma o rifiuta.

Regole controllate

- Date coerenti
- Nessuna sovrapposizione
- Non il proprio alloggio
- Ospiti entro la capienza
- Prezzo = notti × prezzo a notte
- Date non bloccate dall'host

Come è fatto

POST /api/prenotazioni → PrenotazioneService.crea

```
p.dataCheckin < :checkout
AND p.dataCheckout > :checkin
```

Feedback della vostra esperienza

Cosa abbiamo imparato

- Progettare a strati
- Regole di business tracciabili
- Sicurezza con JWT
- Frontend, backend e DB insieme

Difficoltà incontrate

- Un progetto per tre esami
- Date e posti senza incoerenze
- Lavorare in due su molto codice

Cosa miglioreremmo

- Più test automatici
- Paginazione dei risultati
- Messaggi tra host e guest
- Pagamenti reali